

Intermediate Machine Learning: Assignment 4

Deadline

Assignment 4 is due Thursday, November 20 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this [converter notebook](#).

Topics

- Graph kernels and Laplacians
- Graph neural networks
- Policy gradient algorithms
- Deep Q-learning

This assignment will also help to solidify your Python skills. In particular, problem 2 uses PyTorch, whereas many of our implementations have used Tensorflow.

Acknowledgement of Help

I benefitted greatly from discussing the problems with my classmate, Jacey. Also, I used GPT-5 for assistance with the code, mostly for aesthetics and efficiency.

Problem 1: Graph kernels (20 points)

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as img
import sklearn
import random
from numpy.linalg import inv
%matplotlib inline
```

```
In [2]: # Helper functions for third part of exercise

def rgb2gray(rgb):
    """Function to turn RGB images in greyscale images."""
    return np.dot(rgb[..., :3], [0.2989, 0.5870, 0.1140])

def grid_adj(rows, cols):
    """Function that creates the adjacency matrix of
    a grid graph with predefined amount of rows and columns."""
    M = np.zeros([rows*cols, rows*cols])
```

```

for r in np.arange(rows):
    for c in np.arange(cols):
        i = r*cols + c
        if c > 0:
            M[i-1, i] = M[i, i-1] = 1
        if r > 0:
            M[i-cols, i] = M[i, i-cols] = 1
return M

```

The graph Laplacian for a weighted graph on n nodes is defined as

$$L = D - W$$

where W is an $n \times n$ symmetric matrix of positive edge weights, with $W_{ij} = 0$ if (i, j) is not an edge in the graph, and D is the diagonal matrix with $D_{ii} = \sum_{j=1}^n W_{ij}$. This generalizes the definition of the Laplacian used in class, where all of the edge weights are one.

1. Show that L is a Mercer kernel, by showing that L is symmetric and positive-semidefinite.
2. In graph neural networks we define polynomial filters of the form

$$P = a_0 I + a_1 L + a_2 L^2 + \dots a_d L^d$$

where L is the Laplacian and $a_0, \dots, a_d$ are parameters, corresponding to the filter parameters in standard convolutional neural networks.

If each $a_i \geq 0$ is non-negative, show that P is also a Mercer kernel.

1.1 WTS L is a Mercer kernel
 $L = D - W$

i) L is symmetric.

Since W is symmetric and D is diagonal,
 $L = D - W$ is also symmetric.

$$W_{ij} = W_{ji}, D_{ij} = 0 \quad \forall i \neq j$$

$$\Rightarrow L_{ij} = L_{ji}$$

ii) L is psd.

Let $x \in \mathbb{R}^n$.

$$x'Dx = \sum_{i=1}^n D_{ii} x_i^2 = \sum_{i,j} W_{ij} x_i^2$$

$$\text{and } x'Wx = \sum_{i,j} x_i W_{ij} x_j$$

$$\Rightarrow x' L x = \sum_{i,j} W_{ij} x_i^2 - \sum_{i,j} x_i W_{ij} x_j$$

$$\text{Since } W_{ij} = W_{ji}, \quad x' L x = \frac{1}{2} \sum_{i,j} W_{ij} (x_i^2 + x_j^2 - 2x_i x_j)$$

$$= \frac{1}{2} \sum_{i,j} W_{ij} (x_i - x_j)^2 \geq 0$$

$$\text{Since } W_{ij} \geq 0 \text{ and } (x_i - x_j)^2 \geq 0$$

$\therefore L$ is a Mercer kernel, since it is symmetric psd.

$$1.2) \quad P = a_0 I + a_1 L + a_2 L^2 + \dots + a_d L^d$$

WTS: $a_i \geq 0 \Rightarrow P$ is Mercer kernel

i) P is symmetric.

L is symmetric, so L^k is symmetric.

I is also symmetric, so $a_i L^i$ is symmetric.

Sum of symmetric matrices is symmetric, so

P is symmetric.

ii) P is psd.

$$x'Px = x'a_0Ix + x'a_1Lx + x'a_2L^2x + \dots + x'a_dL^dx$$

$$x'a_0Ix = \sum_{i=1}^n a_0 x_i^2 \geq 0$$

$$x'a_1Lx = a_1 x' Lx \geq 0 \text{ since } x' Lx \geq 0 \text{ \& } a_1 \geq 0$$

For $x'a_k L^k x$, suppose $k=2m$ is even. Then

$$x'a_k L^{2m} x = a_k (L^m x)' (L^m x) = a_k \|L^m x\|_2^2 \geq 0$$

Suppose $k=2m+1$ is odd. Then

$$x'a_k L^{2m+1} x = a_k (L^m x)' L (L^m x)$$

Since $L^m x$ is some vector y and L is psd,

$$x'a_k L^{2m+1} x = a_k y' L y \geq 0$$

$\therefore x'Px \geq 0$, so P is psd.

$\therefore P$ is a Mercer kernel

3. This polynomial filter has many applications. A handful of these applications are based on the fact that, given a graph with a signal x , the value of $x^T L x$ will be low in case the signal is smooth (i.e. smooth transitions of x between neighboring nodes). A large $x^T L x$ means that we have a rough graph signal (i.e. a lot of jumps in x between neighboring nodes).

An interesting application that uses this property is the so-called image inpainting process, where an image is seen as grid graph. Image inpainting tries to restore a corrupted image by smoothing out the neighboring pixel values. In this problem we corrupt an image by turning off (i.e. making the pixel value equal to zero) a certain portion of the pixels. Your goal will

be to restore the corrupted image and hence recreate the original image.

First, let's corrupt an image by turning off a portion of the pixels. For this exercise, we choose to turn off 30% of the pixels. The result is shown below. Try to understand the code, as some variables might be interesting for your work.

The image "Yale_Bulldogs.jpg" can be found in this GitHub repo: <https://github.com/YData123/sds365-fa25/tree/main/assignments/assn4>

```
In [3]: from google.colab import drive
drive.mount('/content/drive')
```

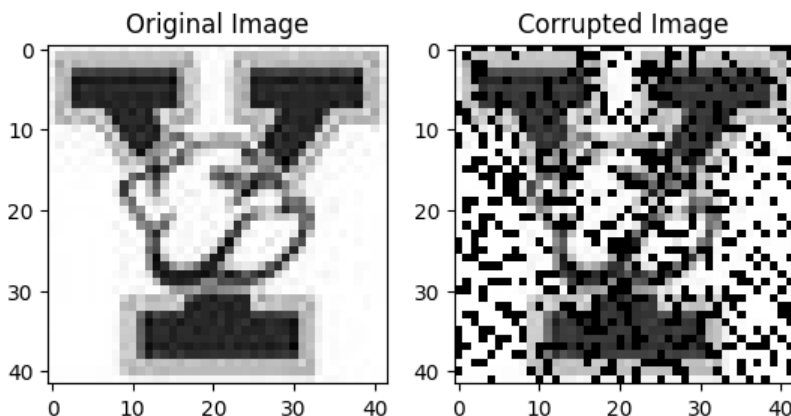
Mounted at /content/drive

```
In [4]: # Normalize the pixels of the original image
image = img.imread("/content/drive/MyDrive/Yale_Bulldogs.jpg") / 255
# Turn picture into greyscale
gray_image = rgb2gray(image)
height_img = gray_image.shape[0]
width_img = gray_image.shape[1]

# Turn off (value 0) certain pixels
fraction_off = int(0.30*height_img*width_img)
mask = np.ones(height_img*width_img, dtype=int)
# Set the first fraction of pixels off
mask[:fraction_off] = 0
# Shuffle to create randomness
np.random.shuffle(mask)
# Multiply the original image by the reshaped mask
mask = np.reshape(mask, (height_img, width_img))
corrupted_image = np.multiply(mask, gray_image)

fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.imshow(gray_image, cmap=plt.get_cmap('gray'))
ax1.set_title("Original Image")
ax2.imshow(corrupted_image, cmap=plt.get_cmap('gray'))
ax2.set_title("Corrupted Image")

plt.show()
```



Inpainting missing pixel values can be formulated as the following optimization problem:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \{ \|\mathbf{y} - \mathbf{M}\mathbf{x}\|_2^2 + \alpha \mathbf{x}^T \mathbf{P} \mathbf{x} \}$$

where $\mathbf{y} \in \mathbb{R}^n$ (n being the total amount of pixels) is the corrupted graph signal (with missing pixel values being 0) and α is a regularization (smoothing) parameter that controls for smoothness of the graph. $\mathbf{P}$ is the polynomial filter based on the laplacian $\mathbf{L}$. Finally, $\mathbf{M} \in \mathbb{R}^{n \times n}$ is a diagonal matrix that satisfies:

$$\mathbf{M}(i, i) = \begin{cases} 1, & \text{if } \mathbf{y}(i) \text{ is observed} \\ 0, & \text{if } \mathbf{y}(i) \text{ is corrupted} \end{cases}$$

The optimization problem tries to find an $\mathbf{x}$ that matches the observed values in $\mathbf{y}$, and at the same time tries to be smooth

on the graph. Start with deriving a closed form solution of this optimization problem:

$$\underline{1.31} \quad \min_{x \in \mathbb{R}^n} \{ \|y - Mx\|_2^2 + \alpha x' P x \}, \quad y \in \mathbb{R}^n, M \in \mathbb{R}^{n \times n} \text{ diagonal}$$

$$M_{ii} = \begin{cases} 1 & : y_i \text{ obs.} \\ 0 & : y_i \text{ corrupted} \end{cases}$$

$$\begin{aligned} f(x) &= \|y - Mx\|_2^2 + \alpha x' P x = (y - Mx)'(y - Mx) + \alpha x' P x \\ &= y'y - 2x'M'y + x'M'Mx + \alpha x' P x \end{aligned}$$

$$\text{FOC: } 0 = \nabla f(x) = -2M'y + 2x'M'M + 2\alpha Px$$

$$\text{Since } M \text{ diag, } M' = M, M'M = M^2 \text{ since } M_{ii} = \begin{cases} 1 \\ 0 \end{cases}$$

$$\Rightarrow 0 = 2(-My + xM + \alpha Px)$$

$$0 = x(M + \alpha P) - My$$

$$\Rightarrow x^* = (M + \alpha P)^{-1} My$$

Next, let's restore our image. To keep things simple, let's say we already trained the polynomial filter $\mathbf{P}$ of degree 2 and we found the following weights:

$$\mathbf{P} = \mathbf{L} + 0.05 \mathbf{L}^2$$

Fill in the following lines of code and show your reconstructed images next to the corrupted image. Assume that the weights on the graph edges are equal to 1.

```
In [6]: # Corrupted graph signal
y = corrupted_image.reshape(-1)

# Diagonal matrix defined as above
M = np.diag(mask.reshape(-1))

# Adjacency matrix of the graph (by using the helper function)
A = grid_adj(height_img, width_img)

# Diagonal matrix defined as above
D = np.diag(A.sum(axis = 1))

# Graph Laplacian defined as above
L = D-A

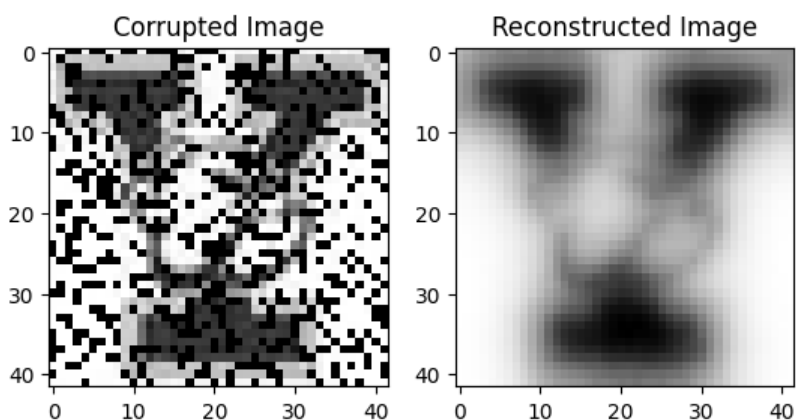
# Polynomial filter defined as above
P = L + 0.05 * (L@L)
```

```
In [7]: # closed form solution you derived above
alpha = 1
x = np.linalg.inv(M + alpha * P) @ (M @ y)
```

```
In [8]: # Try to experiment with different alpha values
alpha = 5
x = np.linalg.inv(M + alpha * P) @ (M @ y)
reconstructed_image = np.reshape(x, (height_img, width_img))

fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.imshow(corrupted_image, cmap=plt.get_cmap('gray'))
ax1.set_title("Corrupted Image")
ax2.imshow(reconstructed_image, cmap=plt.get_cmap('gray'))
ax2.set_title("Reconstructed Image")

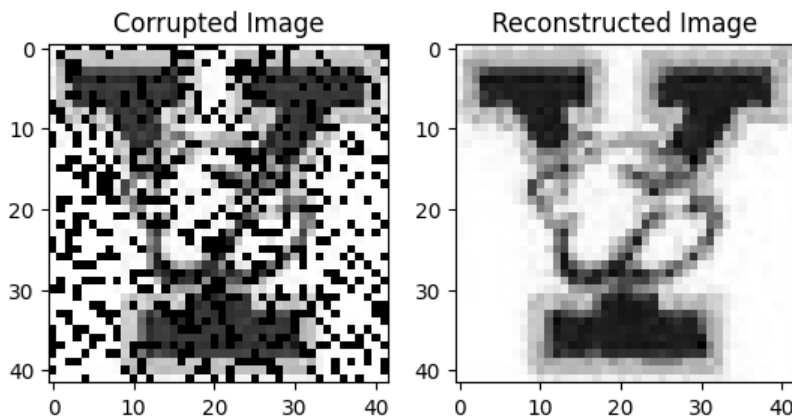
plt.show()
```



```
In [9]: # Try to experiment with different alpha values
alpha = 0.05
x = np.linalg.inv(M + alpha * P) @ (M @ y)
reconstructed_image = np.reshape(x, (height_img, width_img))

fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.imshow(corrupted_image, cmap=plt.get_cmap('gray'))
ax1.set_title("Corrupted Image")
ax2.imshow(reconstructed_image, cmap=plt.get_cmap('gray'))
ax2.set_title("Reconstructed Image")

plt.show()
```



4. Discuss the influence of the smoothing parameter α in the optimization problem above. What happens for very large and very low values of α ? Finally, discuss the degree of our polynomial function $\mathbf{P}$. What happens if we would choose a large degree?

1.4 Response

I tried a few different values of α to experiment with its effect; two representative values are shown above. As we can see, higher alpha results in more smoothing. We get less discrete jumps from one color to another, but we also lose some of the contrast that was present in the original picture, causing our recreation to look blurry. With low alpha, by contrast, we see that we can keep the contrast but at the cost of some imperfections in the recreation. For instance, some of the grays are slightly darker than they should be. In the end, smoothness (i.e., alpha) is a trade-off. With alpha too high, we get too much smoothing and blurriness; with alpha too low, we get not enough smoothing, a noisy reconstruction with corrupted pixels remaining.

For the polynomial function $\mathbf{P}$, the degree of the polynomial controls the "distance" of the smoothing across the image. For a low-degree polynomial, we will get local smoothing, mostly using immediate neighbors. With a higher-degree polynomial, we will get a smoother interpolation across missing areas of the image, but we could get oversmoothing again, as with large alpha.

Problem 2: Protein classification using GNNs (20 points)

In this problem, we focus on a **graph classification** problem using graph neural networks. Specifically, we employ GNNs to generate graph embeddings (a vector representation for each graph), which are then passed through a linear layer for classification. A common graph classification task is molecular property prediction, where molecules are represented as graphs derived from their chemical structures, and the goal is to predict their properties.

Q1: What are some advantages of using GNN and graph representations for molecular property prediction compared to using a fully connected neural networks with tabular data? (5 points)

2.1 Response

Advantages of using GNN and graphs for molecular property prediction:

- **Natural structure.** It seems that the biggest advantage of using graphs for molecules is that we can use the structure of the setting to our advantage. Molecules can naturally be represented by graphs, with the atoms being nodes and the bonds between them being the edges (although I'm not super familiar with chemistry, so I'm sure this is an oversimplification).
- **Order invariance.** Another advantage is that using graphs (and graph operations) are invariant to permutations / ordering, which means we don't need to worry about how the atoms are indexed.
- **Generalization.** Finally, using a GNN will allow the model to share weights across edges/nodes, allowing for fewer overall parameters. I believe this should help us protect against overfitting; i.e., we should be able to generalize better and learn more efficiently.

Here, we'll use the PROTEINS dataset from [TUDataset](#) and our task is to **train a model to predict if the protein is an enzyme or not**.

```
In [10]: import os
import torch
os.environ['TORCH'] = torch.__version__
print(torch.__version__)

!pip install -q torch-scatter -f https://data.pyg.org/whl/torch-${TORCH}.html
!pip install -q torch-sparse -f https://data.pyg.org/whl/torch-${TORCH}.html
!pip install -q git+https://github.com/pyg-team/pytorch_geometric.git

import torch
from torch_geometric.datasets import TUDataset
from torch_geometric.utils import to_networkx
import networkx as nx
import matplotlib.pyplot as plt

2.8.0+cu126
10.9/10.9 MB 82.8 MB/s eta 0:00:00
5.2/5.2 MB 93.9 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for torch-geometric (pyproject.toml) ... done
```

```
In [11]: dataset = TUDataset(root='data/TUDataset', name='PROTEINS')

# general graph information about the whole datasets
print()
print(f'Dataset: {dataset}:')
print('=====')
print(f'Number of graphs: {len(dataset)}')
print(f'Number of features: {dataset.num_features}')
print(f'Number of classes: {dataset.num_classes}')
```

```
Downloading https://www.chrsmrrs.com/graphkerneldatasets/PROTEINS.zip
Processing...
Dataset: PROTEINS(1113):
=====
Number of graphs: 1113
Number of features: 3
Number of classes: 2
Done!
```

As shown above, the dataset contains 1,113 proteins with binary labels, where 1 denotes an enzyme and 0 denotes a non-enzyme. We next examine and visualize the first graph in the dataset to gain a better understanding of its structure.

```
In [12]: # taking a closer look at the first graph
data = dataset[0]

print()
print(data)
print('=====')

# Gather some statistics about the first graph.
print(f'Number of nodes: {data.num_nodes}')
print(f'Number of edges: {data.num_edges}')
print(f'Average node degree: {data.num_edges / data.num_nodes:.2f}')
print(f'Has isolated nodes: {data.has_isolated_nodes()}')
print(f'Has self-loops: {data.has_self_loops()}')
print(f'Is undirected: {data.is_undirected()}')

Data(edge_index=[2, 162], x=[42, 3], y=[1])
=====
Number of nodes: 42
Number of edges: 162
Average node degree: 3.86
Has isolated nodes: False
Has self-loops: False
Is undirected: True
```

The first graph contains 42 nodes each with 3 features and has 162 edges. A visualization of this graph is provided below.

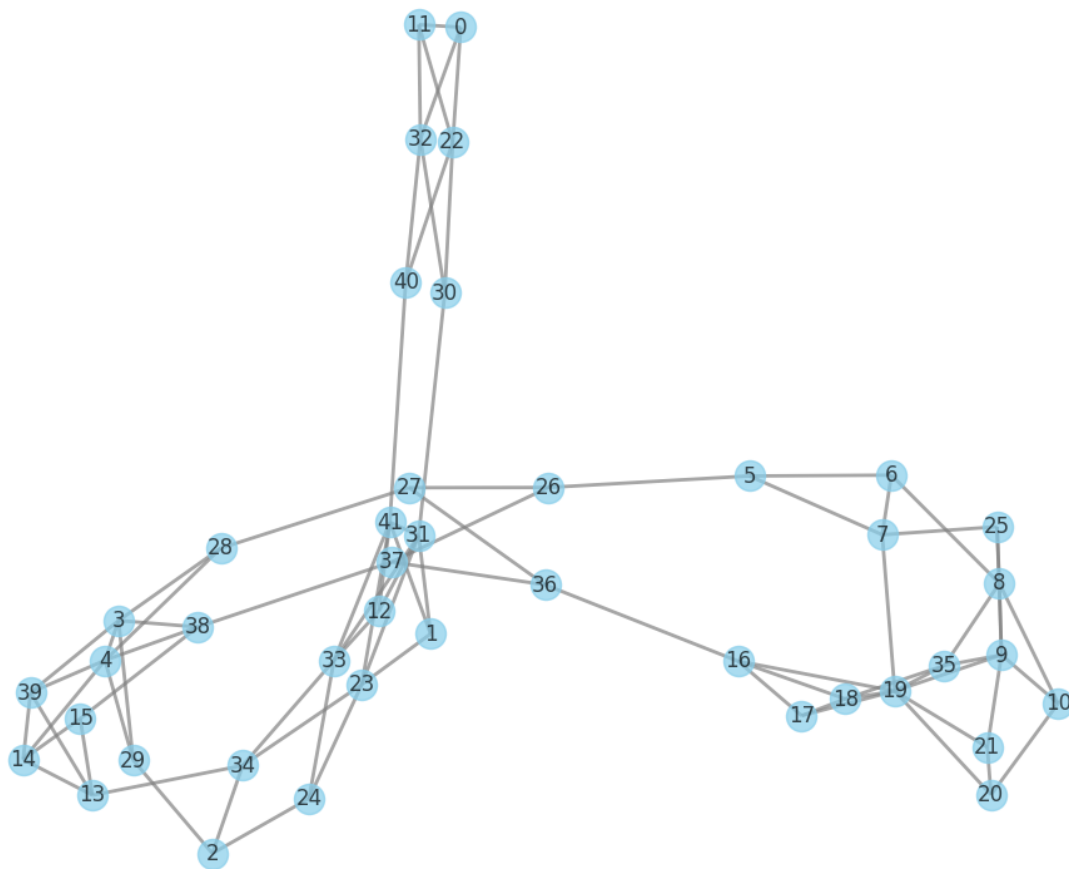
```
In [13]: # Convert to NetworkX using the built-in utility
G = to_networkx(data, to_undirected=True)

# Visualization
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)

nx.draw(G, pos,
        node_color='skyblue',
        node_size=300,
        with_labels=True,
        width=2,
        edge_color='gray',
        alpha=0.7)

plt.title(f"Protein Graph 0, Class: {'Enzyme' if data.y == 1 else 'Non-Enzyme' }", fontsize=15)
plt.show()
```

Protein Graph 0, Class: Non-Enzyme



In the following, similar to all machine learning model training pipeline, we split the dataset, batch them, and start our model building process.

```
In [14]: torch.manual_seed(12345)
dataset = dataset.shuffle()

train_dataset = dataset[:800]
test_dataset = dataset[800:]

print(f'Number of training graphs: {len(train_dataset)}')
print(f'Number of test graphs: {len(test_dataset)}')

from torch_geometric.loader import DataLoader
```

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Number of training graphs: 800
Number of test graphs: 313

A GNN model for graph classification tasks usually come with the following structure:

1. Embed each node by performing multiple rounds of message passing using layers such as Graph Convolutional Layer (more on this later)
2. Aggregate node embeddings into a unified graph embedding (through pooling methods, such as taking the mean, max, sum)
3. Train a final classifier on the graph embedding

A bit more on Graph Convolutional Layers (GCN): The mathematical operation GCN performs are described in detail [here](#).

In simple terms, during a GCN operation, each node's new representation is obtained by aggregating (typically through a weighted average) the current representations of itself and its neighbors, followed by a linear transformation. As in fully connected neural networks, an activation function is usually applied after each GCN layer.

Its node-wise formulation is given by:

$$\mathbf{x}'_i = \Theta^T \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{e_{j,i}}{\sqrt{\hat{d}_j \hat{d}_i}} \mathbf{x}_j$$

Weighted Average

with $\hat{d}_i = 1 + \sum_{j \in \mathcal{N}(i)} e_{j,i}$, where $e_{j,i}$ denotes the edge weight from source node j to target node i (default: 1.0)

Q2: Fill in the necessary code for GCN forward passing based on the description above and the example for the first layer (5 points)

```
In [15]: from torch.nn import Linear
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.nn import global_mean_pool

class GCN(torch.nn.Module):
    def __init__(self, hidden_channels):
        super(GCN, self).__init__()
        torch.manual_seed(12345)
        self.conv1 = GCNConv(dataset.num_node_features, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.conv3 = GCNConv(hidden_channels, hidden_channels)
        self.lin = Linear(hidden_channels, dataset.num_classes)

    def forward(self, x, edge_index, batch):
        # 1. Obtain node embeddings
        x = self.conv1(x, edge_index) # first GCN layer operation
        x = x.relu() # activation function
        #####
        # write your code here: how do we do GCN operation for the second and third layer
        # approximately 4 lines of code (aggregation + activation)

        x = self.conv2(x, edge_index)
        x = x.relu()
        x = self.conv3(x, edge_index)
        x = x.relu()

        #####

        # 2. Pooling layer
        x = global_mean_pool(x, batch) # turn node embedding into graph embedding by taking the mean of all nodes

        # 3. Apply a final classifier
        x = F.dropout(x, p=0.5, training=self.training)
```

```

        x = self.lin(x)

    return x

model = GCN(hidden_channels=64)
print(model)

GCN(
  (conv1): GCNConv(3, 64)
  (conv2): GCNConv(64, 64)
  (conv3): GCNConv(64, 64)
  (lin): Linear(in_features=64, out_features=2, bias=True)
)

```

Q3: After generating graph embeddings, graph classification follows the standard classification pipeline. For our setup, what loss function is appropriate? Briefly explain your choice. Fill in your chosen loss function in the second line of the following code block. Feel free to check the loss function provided in torch [here](#). (5 Points)

2.3 Response

The final layer of the GCN is the linear layer which outputs a vector of (unnormalized) logits. We want probabilities, so using the **cross entropy loss** is appropriate here because it first converts the logits into probabilities using the softmax function, then computes the negative log-likelihood.

```

In [16]: optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
          ## fill in your loss function here
          criterion = torch.nn.CrossEntropyLoss()

def train():
    model.train()

    for data in train_loader: # Iterate in batches over the training dataset.
        out = model(data.x, data.edge_index, data.batch) # Perform a single forward pass.
        loss = criterion(out, data.y) # Compute the loss.
        loss.backward() # Derive gradients.
        optimizer.step() # Update parameters based on gradients.
        optimizer.zero_grad() # Clear gradients.

def test(loader):
    model.eval()

    correct = 0
    for data in loader: # Iterate in batches over the training/test dataset.
        out = model(data.x, data.edge_index, data.batch)
        pred = out.argmax(dim=1) # Use the class with highest probability.
        correct += int((pred == data.y).sum()) # Check against ground-truth labels.
    return correct / len(loader.dataset) # Derive ratio of correct predictions.

train_accs = []
test_accs = []

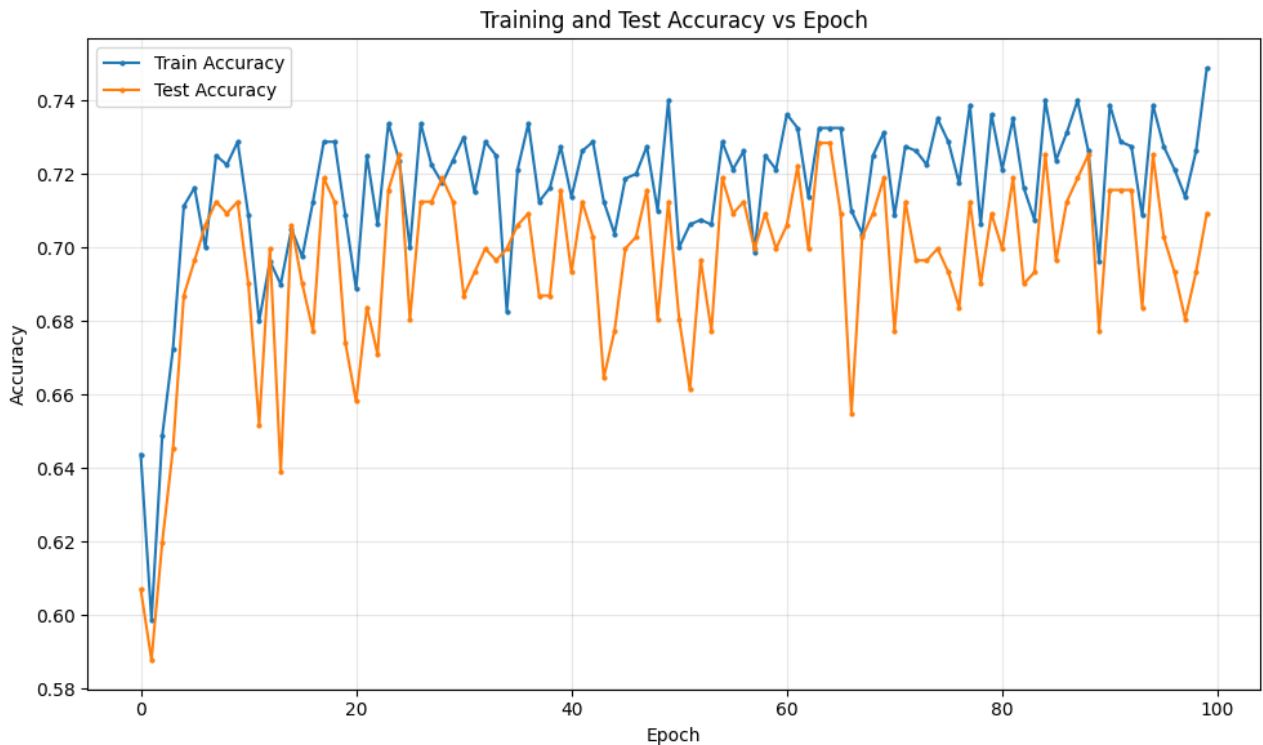
for epoch in range(1, 101):
    train()
    train_acc = test(train_loader)
    test_acc = test(test_loader)
    train_accs.append(train_acc)
    test_accs.append(test_acc)
    print(f'Epoch: {epoch:03d}, Train Acc: {train_acc:.4f}, Test Acc: {test_acc:.4f}')

import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(range(len(train_accs)), train_accs, label='Train Accuracy', marker='o', markersize=2)
plt.plot(range(len(train_accs)), test_accs, label='Test Accuracy', marker='o', markersize=2)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Test Accuracy vs Epoch')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Epoch: 001, Train Acc: 0.6438, Test Acc: 0.6070
Epoch: 002, Train Acc: 0.5988, Test Acc: 0.5879
Epoch: 003, Train Acc: 0.6488, Test Acc: 0.6198
Epoch: 004, Train Acc: 0.6725, Test Acc: 0.6454
Epoch: 005, Train Acc: 0.7113, Test Acc: 0.6869
Epoch: 006, Train Acc: 0.7163, Test Acc: 0.6965
Epoch: 007, Train Acc: 0.7000, Test Acc: 0.7061
Epoch: 008, Train Acc: 0.7250, Test Acc: 0.7125
Epoch: 009, Train Acc: 0.7225, Test Acc: 0.7093
Epoch: 010, Train Acc: 0.7288, Test Acc: 0.7125
Epoch: 011, Train Acc: 0.7087, Test Acc: 0.6901
Epoch: 012, Train Acc: 0.6800, Test Acc: 0.6518
Epoch: 013, Train Acc: 0.6963, Test Acc: 0.6997
Epoch: 014, Train Acc: 0.6900, Test Acc: 0.6390
Epoch: 015, Train Acc: 0.7050, Test Acc: 0.7061
Epoch: 016, Train Acc: 0.6975, Test Acc: 0.6901
Epoch: 017, Train Acc: 0.7125, Test Acc: 0.6773
Epoch: 018, Train Acc: 0.7288, Test Acc: 0.7188
Epoch: 019, Train Acc: 0.7288, Test Acc: 0.7125
Epoch: 020, Train Acc: 0.7087, Test Acc: 0.6741
Epoch: 021, Train Acc: 0.6887, Test Acc: 0.6581
Epoch: 022, Train Acc: 0.7250, Test Acc: 0.6837
Epoch: 023, Train Acc: 0.7063, Test Acc: 0.6709
Epoch: 024, Train Acc: 0.7338, Test Acc: 0.7157
Epoch: 025, Train Acc: 0.7238, Test Acc: 0.7252
Epoch: 026, Train Acc: 0.7000, Test Acc: 0.6805
Epoch: 027, Train Acc: 0.7338, Test Acc: 0.7125
Epoch: 028, Train Acc: 0.7225, Test Acc: 0.7125
Epoch: 029, Train Acc: 0.7175, Test Acc: 0.7188
Epoch: 030, Train Acc: 0.7238, Test Acc: 0.7125
Epoch: 031, Train Acc: 0.7300, Test Acc: 0.6869
Epoch: 032, Train Acc: 0.7150, Test Acc: 0.6933
Epoch: 033, Train Acc: 0.7288, Test Acc: 0.6997
Epoch: 034, Train Acc: 0.7250, Test Acc: 0.6965
Epoch: 035, Train Acc: 0.6825, Test Acc: 0.6997
Epoch: 036, Train Acc: 0.7212, Test Acc: 0.7061
Epoch: 037, Train Acc: 0.7338, Test Acc: 0.7093
Epoch: 038, Train Acc: 0.7125, Test Acc: 0.6869
Epoch: 039, Train Acc: 0.7163, Test Acc: 0.6869
Epoch: 040, Train Acc: 0.7275, Test Acc: 0.7157
Epoch: 041, Train Acc: 0.7137, Test Acc: 0.6933
Epoch: 042, Train Acc: 0.7262, Test Acc: 0.7125
Epoch: 043, Train Acc: 0.7288, Test Acc: 0.7029
Epoch: 044, Train Acc: 0.7125, Test Acc: 0.6645
Epoch: 045, Train Acc: 0.7037, Test Acc: 0.6773
Epoch: 046, Train Acc: 0.7188, Test Acc: 0.6997
Epoch: 047, Train Acc: 0.7200, Test Acc: 0.7029
Epoch: 048, Train Acc: 0.7275, Test Acc: 0.7157
Epoch: 049, Train Acc: 0.7100, Test Acc: 0.6805
Epoch: 050, Train Acc: 0.7400, Test Acc: 0.7125
Epoch: 051, Train Acc: 0.7000, Test Acc: 0.6805
Epoch: 052, Train Acc: 0.7063, Test Acc: 0.6613
Epoch: 053, Train Acc: 0.7075, Test Acc: 0.6965
Epoch: 054, Train Acc: 0.7063, Test Acc: 0.6773
Epoch: 055, Train Acc: 0.7288, Test Acc: 0.7188
Epoch: 056, Train Acc: 0.7212, Test Acc: 0.7093
Epoch: 057, Train Acc: 0.7262, Test Acc: 0.7125
Epoch: 058, Train Acc: 0.6987, Test Acc: 0.6997
Epoch: 059, Train Acc: 0.7250, Test Acc: 0.7093
Epoch: 060, Train Acc: 0.7212, Test Acc: 0.6997
Epoch: 061, Train Acc: 0.7362, Test Acc: 0.7061
Epoch: 062, Train Acc: 0.7325, Test Acc: 0.7220
Epoch: 063, Train Acc: 0.7137, Test Acc: 0.6997
Epoch: 064, Train Acc: 0.7325, Test Acc: 0.7284
Epoch: 065, Train Acc: 0.7325, Test Acc: 0.7284
Epoch: 066, Train Acc: 0.7325, Test Acc: 0.7093
Epoch: 067, Train Acc: 0.7100, Test Acc: 0.6550
Epoch: 068, Train Acc: 0.7037, Test Acc: 0.7029
Epoch: 069, Train Acc: 0.7250, Test Acc: 0.7093
Epoch: 070, Train Acc: 0.7312, Test Acc: 0.7188
Epoch: 071, Train Acc: 0.7087, Test Acc: 0.6773
Epoch: 072, Train Acc: 0.7275, Test Acc: 0.7125
Epoch: 073, Train Acc: 0.7262, Test Acc: 0.6965
Epoch: 074, Train Acc: 0.7225, Test Acc: 0.6965
Epoch: 075, Train Acc: 0.7350, Test Acc: 0.6997

Epoch: 076, Train Acc: 0.7288, Test Acc: 0.6933
 Epoch: 077, Train Acc: 0.7175, Test Acc: 0.6837
 Epoch: 078, Train Acc: 0.7388, Test Acc: 0.7125
 Epoch: 079, Train Acc: 0.7063, Test Acc: 0.6901
 Epoch: 080, Train Acc: 0.7362, Test Acc: 0.7093
 Epoch: 081, Train Acc: 0.7212, Test Acc: 0.6997
 Epoch: 082, Train Acc: 0.7350, Test Acc: 0.7188
 Epoch: 083, Train Acc: 0.7163, Test Acc: 0.6901
 Epoch: 084, Train Acc: 0.7075, Test Acc: 0.6933
 Epoch: 085, Train Acc: 0.7400, Test Acc: 0.7252
 Epoch: 086, Train Acc: 0.7238, Test Acc: 0.6965
 Epoch: 087, Train Acc: 0.7312, Test Acc: 0.7125
 Epoch: 088, Train Acc: 0.7400, Test Acc: 0.7188
 Epoch: 089, Train Acc: 0.7262, Test Acc: 0.7252
 Epoch: 090, Train Acc: 0.6963, Test Acc: 0.6773
 Epoch: 091, Train Acc: 0.7388, Test Acc: 0.7157
 Epoch: 092, Train Acc: 0.7288, Test Acc: 0.7157
 Epoch: 093, Train Acc: 0.7275, Test Acc: 0.7157
 Epoch: 094, Train Acc: 0.7087, Test Acc: 0.6837
 Epoch: 095, Train Acc: 0.7388, Test Acc: 0.7252
 Epoch: 096, Train Acc: 0.7275, Test Acc: 0.7029
 Epoch: 097, Train Acc: 0.7212, Test Acc: 0.6933
 Epoch: 098, Train Acc: 0.7137, Test Acc: 0.6805
 Epoch: 099, Train Acc: 0.7262, Test Acc: 0.6933
 Epoch: 100, Train Acc: 0.7488, Test Acc: 0.7093



Q4: In GNNs, adding more layers doesn't always improve performance and can sometimes dramatically decrease performance on both training and test sets. You might suspect overfitting, but overfitting wouldn't degrade training performance. This phenomenon is called oversmoothing. Based on the name and the "aggregation + activation" operation performed by each layer, what might cause this issue? (Hint: As we stack more layers, we aggregate information from increasingly distant neighbors, and...) (5 Points)

2.4 Response

As we stack more layers, we aggregate information from increasingly distant neighbors, and eventually every node essentially aggregates information from almost the entire graph. Thus, the repeated averaging causes all the nodes to become more similar (in the limit, converging to the same vector). Thus, when all the nodes create nearly identical representations, the graph embeddings lose any meaningful variation and the classification collapses on even the training set (in addition to the test set, of course).

Problem 3: Positive reinforcement (10 points)

As discussed in class, reinforcement learning using policy gradient methods is based on maximizing the expected total reward

$$J(\theta) = \mathbb{E}_{\theta}[R(\tau)],$$

where the expectation is over the probability distribution over sequences τ through a choice of actions using the policy. This can be rewritten as

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\theta}[R(\tau) \nabla_{\theta} \log p(\tau | \theta)].$$

Approximating this gradient involves computing $\nabla_{\theta} \log \pi_{\theta}(a | s)$ where π_{θ} is the policy.

3.1 Continuous action space with Gaussian policy

Suppose that the action space is continuous and $\pi_{\theta}(a | s)$ is a normal density with mean $\mu_{\theta}(s)$ and variance $\sigma_{\theta}^2(s)$, two outputs of a neural network with input s and parameters θ .

Suppose the outputs of the neural network are given by

$$\begin{aligned}\mu_{\theta}(s) &= \beta_1^T h(s) \\ \sigma_{\theta}^2(s) &= \exp(\beta_2^T h(s))\end{aligned}$$

where $h(s)$ is the vector of neurons in the last layer, immediately before the outputs. Derive explicit expressions for $\nabla_{\beta_1} \log \pi_{\theta}(a | s)$ and $\nabla_{\beta_2} \log \pi_{\theta}(a | s)$.

Explain how these gradients and other gradient terms in $\nabla_{\theta} \log \pi_{\theta}(a | s)$ are used to estimate the policy.

$$\underline{3.1} \quad \pi_{\theta}(a|s) \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s))$$

$$\mu_{\theta}(s) = \beta_1' h(s)$$

$$\sigma_{\theta}^2(s) = \exp(\beta_2' h(s))$$

$$\text{pdf: } \pi_{\theta}(a|s) = (2\pi\sigma_{\theta}^2(s))^{-1/2} \exp\left(-\frac{(a-\mu_{\theta}(s))^2}{2\sigma_{\theta}^2(s)}\right)$$

$$\Rightarrow \log \pi_{\theta}(a|s) = -\frac{1}{2} \log(2\pi\sigma_{\theta}^2(s)) - \frac{(a-\mu_{\theta}(s))^2}{2\sigma_{\theta}^2(s)}$$

$$\nabla_{\beta_1} \mu_{\theta}(s) = h(s)$$

$$\Rightarrow \nabla_{\beta_1} \log \pi_{\theta}(a|s) = \frac{a-\mu_{\theta}(s)}{\sigma_{\theta}^2(s)} h(s)$$

$$\nabla_{\beta_2} \sigma_{\theta}^2(s) = \sigma_{\theta}^2(s) h(s)$$

$$\begin{aligned} \Rightarrow \nabla_{\beta_2} \log \pi_{\theta}(a|s) &= \left(-\frac{1}{2\sigma_{\theta}^2(s)} + \frac{(a-\mu_{\theta}(s))^2}{2\sigma_{\theta}^4(s)}\right) \sigma_{\theta}^2(s) h(s) \\ &= \frac{1}{2} \left(\frac{(a-\mu_{\theta}(s))^2}{\sigma_{\theta}^2(s)} - 1\right) h(s) \end{aligned}$$

We're going to use these formulas by sampling trajectories through the state space and calculating an estimate of the gradient $\hat{\nabla}_{\beta_2} J(\theta) = N^{-1} \sum_s \sum_t G_t \nabla_{\beta_2} \log \pi_{\theta}(a_t|s_t)$, where each s is a different trajectory. So, these formulas tell us how changing the mean and variance of the policies change the likelihood of the sampled actions. Once we estimate the gradient, we can update by ascent: $\beta_i \leftarrow \beta_i + \eta \hat{\nabla}_{\beta_i} J(\theta)$.

3.2 Discrete action space with Softmax policy

Suppose the action space is discrete with K possible actions, and the policy $\pi_{\theta}(a|s)$ is defined using a softmax function over preferences $u_{\theta}(s, a)$:

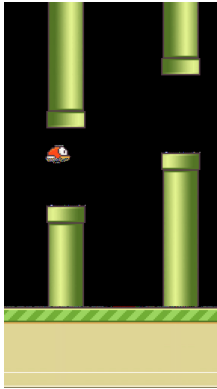
$$\pi_{\theta}(a|s) = \frac{\exp(u_{\theta}(s, a))}{\sum_a \exp(u_{\theta}(s, a))},$$

where $u_\theta(s, a) = \beta^T h(s, a)$, and $h(s, a)$ is a feature vector for state-action pair (s, a) . Derive the expression for $\nabla_\beta \log \pi_\theta(a | s)$.

$$\begin{aligned}
 \underline{3.2]} \quad \pi_\theta(a|s) &= \frac{\exp(u_\theta(s,a))}{\sum_{a'} \exp(u_\theta(s,a'))}, \quad u_\theta(s,a) = \beta^T h(s,a) \\
 \log \pi_\theta(s,a) &= u_\theta(s,a) - \log \sum_{a'} \exp(u_\theta(s,a')) \\
 \nabla_\theta u_\theta(s,a) &= h(s,a) \\
 \Rightarrow \nabla_\theta \log \sum_{a'} \exp(u_\theta(s,a')) &= \frac{\sum_{a'} \exp(u_\theta(s,a')) \nabla_\theta u_\theta(s,a')}{\sum_{a'} \exp(u_\theta(s,a'))} \\
 &= \frac{\sum_{a'} \exp(u_\theta(s,a')) h(s,a')}{\sum_{a'} \exp(u_\theta(s,a'))} = \sum_{a'} \pi_\theta(a'|s) h(s,a') \\
 \Rightarrow \log \pi_\theta(s,a) &= h(s,a) - \sum_{a'} \pi_\theta(a'|s) h(s,a')
 \end{aligned}$$

Problem 4: Deep Q-Learning for Flappy Bird (25 points)

Deep Q-learning was proposed (and patented) by DeepMind and made a big splash when the same deep neural network architecture was shown to be able to surpass human performance on many different Atari games, playing directly from the pixels. In this problem, we will walk you through the implementation of deep Q-learning to learn to play the Flappy Bird game.



The implementation is based these references:

- [DeepLearningFlappyBird](#)
- [Deep Q-Learning for Atari Breakout](#)

We use the `pygame` package to visualize the interaction between the algorithm and the game environment. However, `pygame` is not well supported by Google Colab; we recommend you to run the code for this problem locally. A window will be popped up that displays the game as it progress in real-time (as for the Cartpole demo from class).

This problem is structured as follows:

- Load necessary packages
- Test the visualization of the game, to make sure everything's working
- Process the images to reduce the dimension
- Setup the game history buffer
- Implement the core Q-learning function
- Run the learning algorithm
- Interpret the results

Introduction

The Flappy Bird game is requires a few Python packages. Please install these *as soon as possible*, and notify us of any issues you experience so that we can help. The Python files can also be found on Canvas and in our GitHub repo at <https://github.com/YData123/sds365-fa25/tree/main/assignments/assn4> .

```
In [46]: # %pip install pygame
import numpy as np
import cv2
from collections import deque
import random
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, initializers
import wrapped_flappy_bird as flappy_bird

# In order to write the wrapped_flappy_bird utilities, you'll need to upload the following
# three files to /content on Colab: wrapped_flappy_bird.py, flappy_bird_utils.py, assets.zip.
# Then, unzip the file assets.zip -- you can do this by opening a terminal.
```

The Flappy Bird environment

Interaction with the game environment is carried out through calls of the form

```
(image, reward, terminal) = game.frame_step(action)
```

where the meaning of these variables is as follows:

- `action` : $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ for doing nothing, $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ for "flapping the bird's wings"
- `image` : the image for the next step of the game, of size $(288, 512, 3)$ with three RGB channels
- `reward` : the reward received for taking the action; -1 if an obstacle is hit, 0.1 otherwise.

- `terminal` : True if an obstacle is hit, otherwise False

Now let's take a look at the game interface. First, initiate the game:

```
In [47]: num_actions = 2

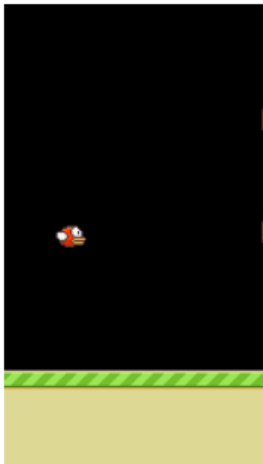
# initiate a game
game = flappy_bird.GameState()

# get the first state by doing nothing
do_nothing = np.zeros(num_actions)
do_nothing[0] = 1
image, reward, terminal = game.frame_step(do_nothing)

import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=(8, 4))
plt.imshow(image.transpose([1, 0, 2]))
plt.axis('off')
plt.title(f'Flappy Bird - Reward: {reward}')
plt.show()

print('shape of image:', image.shape)
print('reward: ', reward)
print('terminal: ', terminal)
```

Flappy Bird - Reward: 0.1



```
shape of image: (288, 512, 3)
reward: 0.1
terminal: False
```

Let's take some random actions and see what happens:

```
In [48]: from IPython.display import clear_output
import time
for i in range(100):

    # choose a random action
    action = np.random.choice(num_actions)

    # create the corresponding one-hot vector
    action_vec = np.zeros(num_actions)
    action_vec[action] = 1

    # take the action and observe the reward and the next state
    image, reward, terminal = game.frame_step(action_vec)

    clear_output(wait=True) # Clear previous frame
    time.sleep(.1)
    plt.imshow(image.transpose([1, 0, 2]))
    plt.axis('off')
    plt.title(f'Frame {i}, Action: {"Flap" if action == 0 else "No Flap"}, Reward: {reward}')
    plt.show()
```

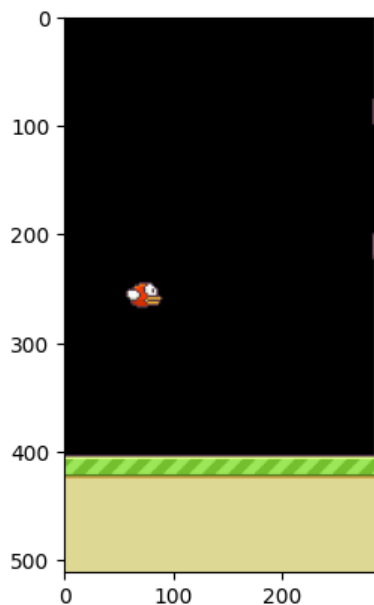
Frame 99, Action: Flap, Reward: 0.1



Are you able to see Flappy moving across the window and crashing into things? Great! If you're having any issues, post to EdD and we'll do our best to help you out.

Here is how we can visualize a frame of the game as an image within a cell.

```
In [49]: # show the image
import matplotlib.pyplot as plt
plt.imshow(image.transpose([1, 0, 2]))
plt.show()
plt.close()
```



Preprocessing the images

Alright, next we need to preprocess the images by converting them to grayscale and resizing them to 80×80 pixels. This will help to reduce the computation, and aid learning. Besides, Flappy is "color blind." (Fun fact: The instructor of this course is also [color vision deficient](#).)

```
In [50]: def resize_gray(frame):
frame = cv2.cvtColor(cv2.resize(frame, (80, 80)), cv2.COLOR_BGR2GRAY)
ret, frame = cv2.threshold(frame, 1, 255, cv2.THRESH_BINARY)
return np.reshape(frame, (80, 80, 1))
```

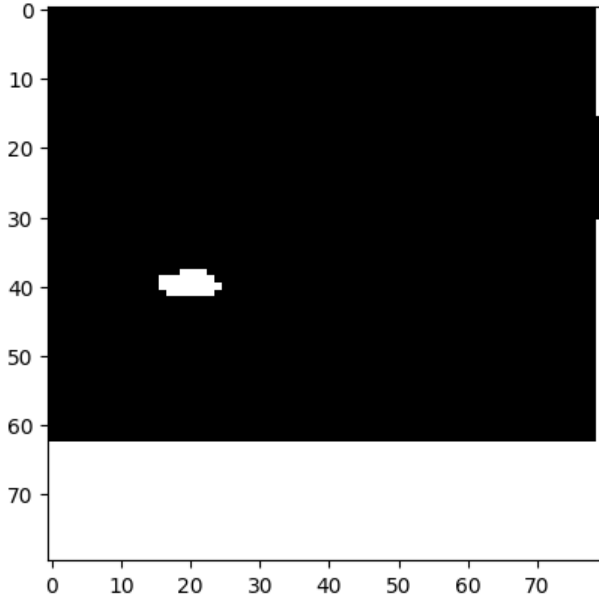
```

image_transformed = resize_gray(image)
print('Shape of the transformed image:', image.shape)

# show the transformed image
_ = plt.imshow(image_transformed.transpose((1, 0, 2)), cmap='gray')

```

Shape of the transformed image: (288, 512, 3)



This shows the preprocessed image for a single frame of the game. In our implementation of Deep Q-Learning, we encode the state by stacking four consecutive frames, resulting in a tensor of shape (80,80,4).

Then, given the `current_state`, and a raw image `image_raw` of size $288 \times 512 \times 3$, we convert the raw image to a $80 \times 80 \times 1$ grayscale image using the code in the previous cell. Then, we remove the first frame of `current_state` and add the new frame, giving a stack of images of size (80, 80, 4).

```

In [51]: def preprocess(image_raw, current_state=None):
# resize and convert to grayscale
image = resize_gray(image_raw)
# stack the frames
if current_state is None:
    state = np.concatenate((image, image, image, image), axis=2)
else:
    state = np.concatenate((image, current_state[:, :, :3]), axis=2)
return state

```

4.1 Explain the game state

Why is the state chosen to be a stack of four consecutive frames rather than a single frame? Give an intuitive explanation.

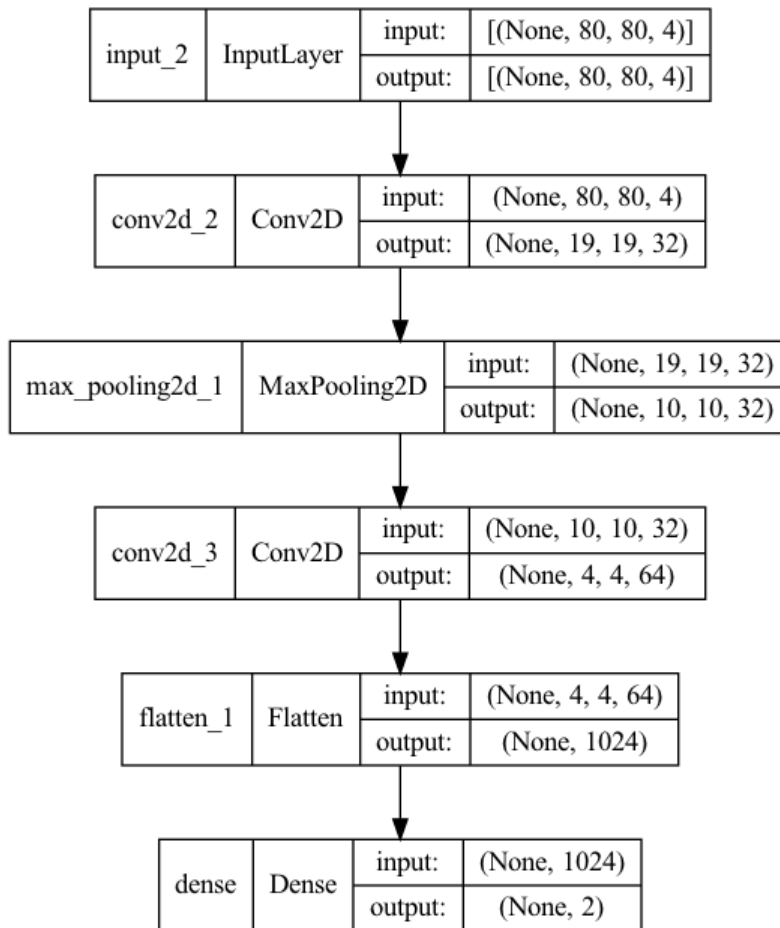
4.1 Response

Basically, a single frame does not contain key information about the environment: namely, the velocity of various parts. For instance, how fast the bird is flying and whether the bird is rising or falling are both key factors affecting the decision maker. By stacking four frames, we can effectively capture this motion/time element.

Constructing the neural network

Now we are ready to construct the neural network for approximating the Q function. Recall that, given input s which is of size $80 \times 80 \times 4$ due to the previous preprocessing, the output of the network should be of size 2, corresponding to the values of $Q(s, a_1)$ and $Q(s, a_2)$ respectively.

Here is the summary of the model we'd like to build:



4.2 Initialize the network

Complete the code in the next cell so that your model architecture matches that in the above picture. Here we specify the initialization of the weights by using `keras.initializers`. Note that we haven't talked about the `strides` argument for CNNs; you can read about stride here: <https://machinelearningmastery.com/padding-and-stride-for-convolutional-neural-networks/>. It's not important to understand this in detail, you just need to choose the number and sizes of the filters to get the shapes to match the specification.

```
In [52]: from tensorflow.keras import initializers
def create_q_model():
    state = layers.Input(shape=(80, 80, 4))

    layer1 = layers.Conv2D(filters=32, kernel_size=(8,8), strides=4, activation="relu",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(state)
    layer2 = layers.MaxPool2D(pool_size=(2,2), strides=2, padding="same")(layer1)
    layer3 = layers.Conv2D(filters=64, kernel_size=(4,4), strides=2, activation="relu",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(layer2)
    layer4 = layers.Flatten()(layer3)
    q_value = layers.Dense(units=2, activation="linear",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(layer4)

    return keras.Model(inputs=state, outputs=q_value)
```

Plot the model summary to make sure that the network is the same as expected.

```
In [53]: model = create_q_model()
print(model.summary())
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 80, 80, 4)	0
conv2d_3 (Conv2D)	(None, 19, 19, 32)	8,224
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 32)	0
conv2d_4 (Conv2D)	(None, 4, 4, 64)	32,832
flatten_1 (Flatten)	(None, 1024)	0
dense_1 (Dense)	(None, 2)	2,050

Total params: 43,106 (168.38 KB)

Trainable params: 43,106 (168.38 KB)

Non-trainable params: 0 (0.00 B)

None

Deep Q-learning

We're now ready to implement the Q-learning algorithm. There are some subtle details in the implementation that you need to sort out. First, recall that the update rule for Q learning is

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r(s, a) + \gamma \cdot \max_{a'} Q(\text{next}(s, a), a') - Q(s, a))$$

where γ is the discount factor and α can be viewed as the step size or learning rate for gradient ascent.

We'll set these as follows:

```
In [54]: gamma = 0.99          # decay rate of past observations
         step_size = 1e-4     # step size
```

Estimation with experience replay

At the beginning of training, we spend 10,000 steps taking random actions, as a means of observing the environment.

We build a replay memory of length 10,000 steps, and every time we update the weights of the network, we sample a batch of size 32 and perform a Q-learning update on this batch.

After we have collected 10,000 steps of new data, we discard the old data, and replace it with the new "experiences."

```
In [55]: observe = 10000      # timesteps to observe before training
         replay_memory = 10000 # number of previous transitions to remember
         batch_size = 32       # size of each batch
```

4.3 Justify the data collection

Why does it make sense to maintain the replay memory of a fixed size instead of including all of the historical data?

4.3 Response

I can think of two key benefits to keeping a fixed memory size:

- **Computational efficiency.** Perhaps most obviously, limiting the memory capacity reduces the computational demand on the model (and thus my computer). Keeping a complete history would require much more space, slow down sampling, and reduce the overall training speed.
- **Relevant training.** Since the agent will improve as the training progresses, the more recent transitions will be most relevant. Keeping an entire history would keep learning from decisions that are less representative of the current (more optimal) behavior and could slow or even destabilize learning.

Exploration vs exploitation

When performing Q-learning, we face the tradeoff between exploration and exploitation. To encourage exploration, a simple strategy is to take a random action at each step with certain probability.

More precisely, for each time step t and state s_t , with probability ϵ , the algorithm takes a random action (wing flap or do nothing), and with probability $1 - \epsilon$ the algorithm takes a greedy action according to $a_t = \arg \max_a Q_\theta(s_t, a)$. Here θ refers to the parameters of our CNN.

```
In [56]: # value of epsilon
epsilon = 0.05
```

4.4 Complete the Q-learning algorithm

Next you will need to complete the Q-learning algorithm by filling in the missing code in the following function. The missing parts include

- Taking a greedy action
- Given a batch of samples $\{(s_t, a_t, r_t, s_{t+1}, \text{terminal}_t)\}_{t \in B}$, computing the corresponding $Q_\theta(s_t, a_t)$.
- Given a batch of samples $\{(s_t, a_t, r_t, s_{t+1}, \text{terminal}_t)\}_{t \in B}$, computing the corresponding updated Q-values

$$\hat{y}(s_t, a_t) = \begin{cases} r_t + \gamma \max_a Q_\theta(s_{t+1}, a), & \text{if } \text{terminal}_t = 0, \\ r_t, & \text{otherwise.} \end{cases}$$

Then, the mean squared error loss for the batch is

$$\frac{1}{|B|} \sum_{t \in B} (\hat{y}(s_t, a_t) - Q_\theta(s_t, a_t))^2.$$

```
In [57]: def dqf_flappy_bird(model, optimizer, loss_function):

    # initiate a game
    game = flappy_bird.GameState()

    # store the previous state, action and transitions
    history_data = deque()

    # get the first observation by doing nothing and preprocess the image
    do_nothing = np.zeros(num_actions)
    do_nothing[0] = 1
    image, reward, terminal = game.frame_step(do_nothing)

    # preprocess to get the state
    current_state = preprocess(image_raw=image)

    # training
    t = 0

    while t < 50000:
        if epsilon > np.random.rand(1)[0]:
            # random action
            action = np.random.choice(num_actions)
        else:
            # compute the Q function
            current_state_tensor = tf.convert_to_tensor(current_state)
            current_state_tensor = tf.expand_dims(current_state_tensor, 0)
            q_value = model(current_state_tensor, training=False)

            # greedy action
            cs = tf.cast(tf.expand_dims(current_state, 0), tf.float32) / 255
            q_value = model(cs, training=False)
            action = int(tf.argmax(q_value[0]).numpy())

        # take the action and observe the reward and the next state
        action_vec = np.zeros([num_actions])
        action_vec[action] = 1
        image_raw, reward, terminal = game.frame_step(action_vec)
        next_state = preprocess(current_state=current_state,
                                image_raw=image_raw)

        # store the observation
```



```

history_data.append((current_state, action, reward, next_state,
                    terminal))
if len(history_data) > replay_memory:
    history_data.popleft() # discard old data

# train if done observing
if t > observe:

    # sample a batch
    batch = random.sample(history_data, batch_size)
    state_sample = np.array([d[0] for d in batch])
    action_sample = np.array([d[1] for d in batch])
    reward_sample = np.array([d[2] for d in batch])
    state_next_sample = np.array([d[3] for d in batch])
    terminal_sample = np.array([d[4] for d in batch])

    # compute the updated Q-values for the samples
    s_next = tf.cast(tf.convert_to_tensor(state_next_sample), tf.float32) / 255
    q_next = model(s_next, training=False)
    max_q_next = tf.reduce_max(q_next, axis=1)
    not_terminal = 1.0 - tf.cast(terminal_sample, tf.float32)
    updated_q_value = tf.cast(reward_sample, tf.float32) + gamma * not_terminal * max_q_next
    updated_q_value = tf.stop_gradient(updated_q_value)

    # train the model on the states and updated Q-values
    with tf.GradientTape() as tape:
        # compute the current Q-values for the samples
        s_cur = tf.cast(tf.convert_to_tensor(state_sample), tf.float32) / 255
        q_all = model(s_cur, training=True)
        action_one_hot = tf.one_hot(action_sample, depth=num_actions, dtype=tf.float32)
        current_q_value = tf.reduce_sum(q_all * action_one_hot, axis=1)

        # compute the loss
        loss = loss_function(updated_q_value, current_q_value)

    # backpropagation
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
else:
    loss = 0

# update current state and counter
current_state = next_state
t += 1

# print info every 500 steps
if t % 500 == 0:
    print(f"STEP {t} | PHASE {'observe' if t<=observe else 'train'}",
          f"| ACTION {action} | REWARD {reward} | LOSS {loss}")

```

You're now ready to play the game! Just run the cell below; do not change the code.

```

In [58]: def playgame():

    #! DO NOT change the random seed !
    np.random.seed(4)

    model = create_q_model()

    # specify the optimizer and loss function
    optimizer = keras.optimizers.Adam(learning_rate=step_size, clipnorm=1.0)
    loss_function = keras.losses.MeanSquaredError()

    # play the game
    dql_flappy_bird(model=model, optimizer=optimizer, loss_function=loss_function)

In [ ]: playgame()

```

STEP 500	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 1000	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 1500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 2000	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 2500	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 3000	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 3500	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 4000	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 4500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 5000	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 5500	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 6000	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 6500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 7000	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 7500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 8000	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 8500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 9000	PHASE observe	ACTION 0	REWARD 0.1	LOSS 0
STEP 9500	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 10000	PHASE observe	ACTION 1	REWARD 0.1	LOSS 0
STEP 10500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.06104866415262222
STEP 11000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.008025054819881916
STEP 11500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.005274229682981968
STEP 12000	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.010992535389959812
STEP 12500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.0044978610239923
STEP 13000	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.003364647040143609
STEP 13500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.004151920322328806
STEP 14000	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.001849387539550662
STEP 14500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0016199108213186264
STEP 15000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.001303308759815991
STEP 15500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.0012540663592517376
STEP 16000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.003389680990949273
STEP 16500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.0008829028811305761
STEP 17000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0014083965215831995
STEP 17500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0028497714083641768
STEP 18000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0009642724180594087
STEP 18500	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.0015190709382295609
STEP 19000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0007801807369105518
STEP 19500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0009666872210800648
STEP 20000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.043498676270246506
STEP 20500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0018409665208309889
STEP 21000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0023034033365547657
STEP 21500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.0042177839204669
STEP 22000	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.005745744798332453
STEP 22500	PHASE train	ACTION 0	REWARD 0.1	LOSS 0.03514164686203003
STEP 23000	PHASE train	ACTION 1	REWARD 0.1	LOSS 0.009761681780219078

4.5 Describe the training

Describe what you see by answering the following questions:

- In the early stage of training (within 2,000 steps in the *explore* phase), describe the behavior of the Flappy Bird. What do you think is the greedy policy given by the estimation of the Q-function in this stage?
- Describe what you see after roughly 5,000 training steps. Do you see any improvement? In particular, compare Flappy's behavior with their behavior in the early stages of training.
- Explain why the performance has improved, by relating to the model design such as the replay memory and the exploration.

4.5 Response

In the early stages of training, we see that the reward stays constant (and low). I take this to mean that the bird is running into the very first pipe every time. We should expect this, since in the beginning the Q values are almost random; moreover, the greedy policy is still using a large ϵ , so the actions are mostly random anyway (regardless of the value function).

After ~5,000 training steps, I still see a low reward, but the loss has fallen quite a bit from the initial training stage. [n.b. Here, I don't actually record, e.g., how many frames the bird survives in a given play, but for my following answer my friend shared with me her results since she was smart and tracked these things]. That being said, we do see that the bird survives longer in some rounds. Specifically, we see an increase in the variation of survival frames, indicating that the bird is making it past not only the first pipe but also past maybe a couple pipes in a row. This seems like good evidence to me of learning.

There are several reasons why the performance improves over the course of the training. First, since we included exploration in the model (both through greedy epsilon actions and initially randomized value functions), there is room for the bird to accidentally make it through the first pipe. The learning comes from generalizing (e.g., being in position to flap up so that the bird goes through the gap, regardless of where the gap in the pipe is). Given a couple happy mistakes, we can update the Q functions slowly but surely so that we start making them happen more often, over the course of thousands of iterations. The epsilon in the greedy action policy also becomes smaller, so that we can exploit what we've learned. At the exploitation stage, the limited memory comes in handy, since we can learn much more from the more recent and more successful runs. Eventually, you get a model that can perform as well as the one linked in the YouTube video.

It takes a long time to fully train the network, so you're not required to complete the training. Here's a [video](#) showing the performance of a well trained DQN.

I ran the model for about half an hour to see the initial changes so I could answer the questions, then I stopped it so I could print my responses.